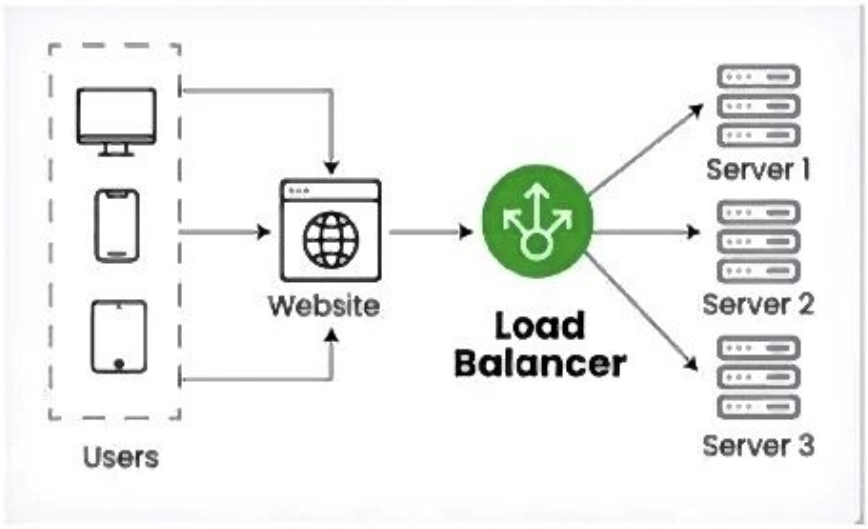
What is Load Balancer & How Load Balancing works?

A load balancer is a networking device or software application that distributes and balances the incoming traffic among the servers to provide high availability, efficient utilization of servers and high performance.



LoadBalancer

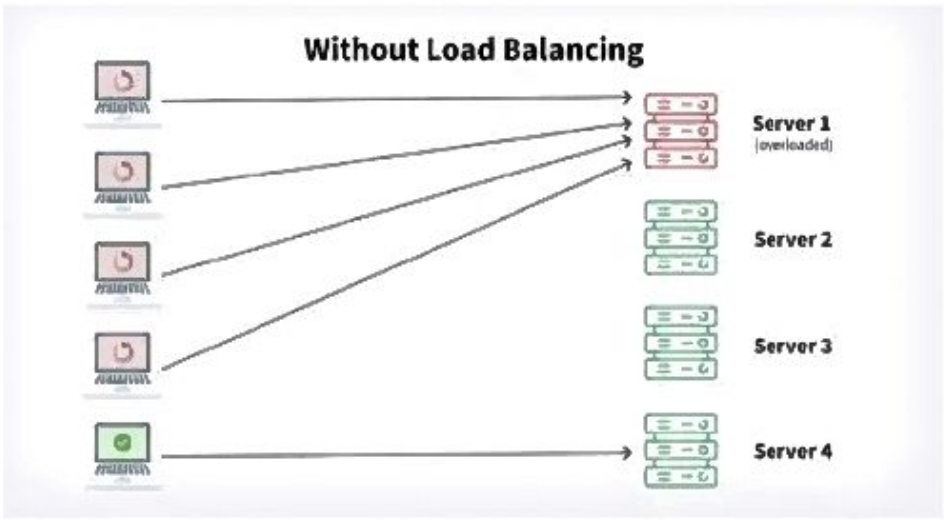
Works as a "traffic cop" routing client requests across all servers

Ensures that no single server bears too many requests, which helps improve the performance, reliability and availability of applications.

Highly used in cloud computing domains, data centers and large-scale web applications where traffic flow needs to be managed.

Issues without Load Balancer?

Several problem will occur without the load balancer, these are:



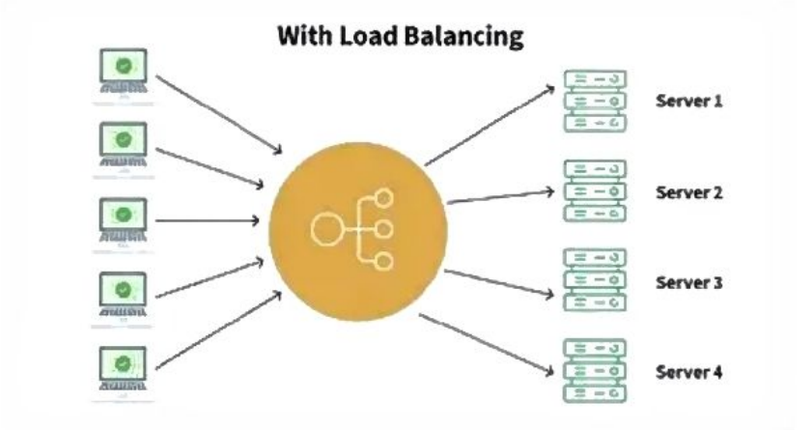
without-load-balancing

Single Point of Failure: If the server goes down or something happens to the server the whole application will be interrupted and it will become unavailable for the users for a certain period. It will create a bad experience for users which is unacceptable for service providers.

Overloaded Servers: There will be a limitation on the number of requests that a web server can handle. If the business grows and the number of requests increases the server will be overloaded.

Limited Scalability: Without a load balancer, adding more servers to share the traffic is complicated. All requests are stuck with one server and adding new servers won't automatically solve the load issue.

Note: These are the problems that are resolved with Load Balancer.



with-load-balancing

Key characteristics of Load Balancers

Traffic Distribution: To keep any one server from becoming overburdened, load balancers divide incoming requests evenly among several servers.

High Availability: Applications' reliability and availability are improved by load balancers, which divide traffic among several servers. The load balancer reroutes traffic to servers that are in good condition in the event that one fails.

Scalability: By making it simple to add servers or resources to meet growing traffic demands, load balancers enable horizontal scaling.

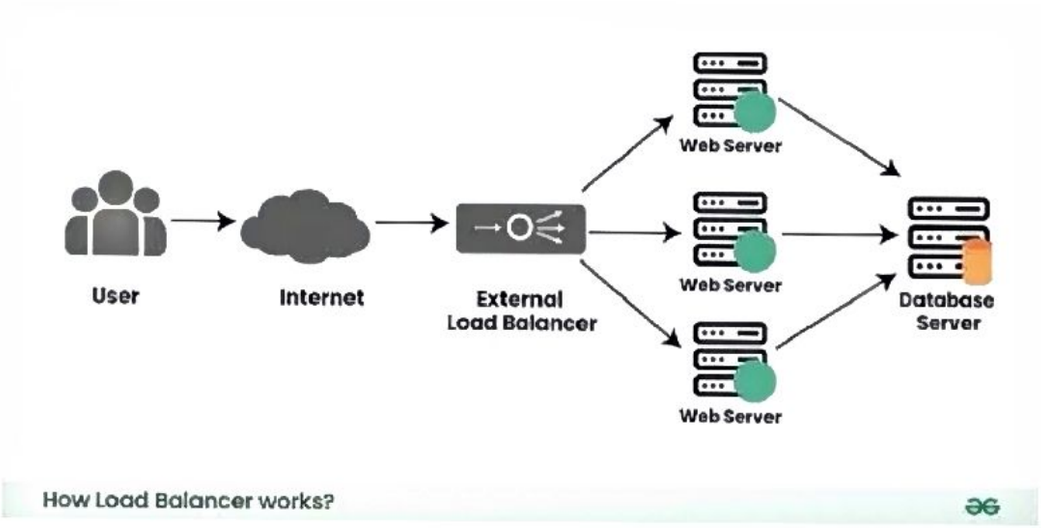
Optimization: Load balancers optimize resource utilization, ensuring efficient use of server capacity and preventing bottlenecks.

Health Monitoring: Load balancers often monitor the health of servers, directing traffic away from servers experiencing issues or downtime.

SSL Termination: Some load balancers can handle SSL/TLS encryption and decryption, offloading this resource-intensive task from servers.

How Load Balancer Works?

How-Load-Balancer-works



Receives Incoming Requests: When users try to access a website or application, their requests first go to the load balancer instead of directly to a server.

Checks Server Health: The load balancer continuously monitors the status of all servers. It checks which servers are healthy and ready to handle requests.

Distributes Traffic: Based on factors like server load, response time or proximity, the load balancer forwards each request to the most appropriate server. This helps avoid any server getting overloaded.

Handles Server Failures: If a server goes down or becomes unresponsive, the load balancer automatically stops sending traffic to that server and redirects it to others that are still functioning properly.

Optimizes Performance: By spreading traffic efficiently and using healthy servers, load balancers improve overall performance and reduce delays.

Benefits of Load Balancer

Better Performance - Distributes traffic across servers so no single server gets overloaded, reducing downtime and improving speed.

Scalability - Works with auto-scaling to add more servers during high traffic and remove them when traffic is low.

Failure Handling - Detects unhealthy servers and redirects traffic to healthy ones, keeping the system available.

Prevents Bottlenecks - Handles sudden spikes in traffic smoothly by spreading requests evenly.

Efficient Resource Use - Ensures all servers share the workload fairly.

Session Persistence - Can maintain user sessions so apps that need continuous sessions (like shopping carts) work properly

Challenges of Load Balancer

Single Point of Failure - If the load balancer itself goes down, it can disrupt traffic flow unless a backup exists.

Cost and Complexity - Good load balancing solutions can be expensive and require proper setup and management.

Configuration Issues - Setting up correctly can be tricky, especially for complex applications.

Extra Overhead - Adds a small delay since every request passes through the load balancer.

SSL Management - Handling encryption (SSL termination) at the balancer can make end-to-end security more complicated.

Types of Load Balancer

Load Balancers distribute incoming network traffic across multiple servers to ensure optimal resource utilization, minimize response time, and prevent server overload.

When it comes to load balancing, three primary types exist : software load balancers, hardware load balancers, and virtual load balancers.

Types of Load Balancer - Based on Configurations

These load balancers are categorized according to how they are set up and managed in a system. They define whether traffic distribution is handled by hardware, software, or cloud-based configurations.

Software Load Balancers

Software load balancers are applications or components that run on general-purpose servers. They are implemented in software, making them flexible and adaptable to various environments.

- The application chooses the first one in the list and requests data from the server.
- If any failure occurs persistently (after a configurable number of retries) and the server becomes unavailable, it discards that server and chooses the other one from the list to continue the process.
- This is one of the cheapest ways to implement load balancing.

Hardware Load Balancers

As the name suggests we use a physical appliance to distribute the traffic across the cluster of networ servers. These load balancers are also known as Layer 4-7 Routers and these are capable of handling all kinds of HTTP, HTTPS, TCP, and UDP traffic.

Hardware load balancers are dedicated devices designed for the sole purpose of managing network traffic. They often come as standalone appliances or modules within networking hardware.

- HLBs can handle a large volume of traffic but it comes with a hefty price tag and it also has limited flexibility.
- If any of the servers don't produce the desired response, it immediately stops sending the traffic to the servers.
- These load balancers are expensive to acquire and configure, which is the reason a lot of service providers use them only as the first entry point for user requests.
- Later the internal software load balancers are used to redirect the data behind the infrastructure wall.

Virtual Load Balancers

A virtual load balancer is a type of load balancing solution implemented as a virtual machine (VM) o software instance within a virtualized environment ,such as data centers utilizing virtualization technologies like VMware, Hyper-V, or KVM. It plays a crucial role in distributing incoming networ traffic across multiple servers or resources to ensure efficient utilization of resources, improve response times, and prevent server overload.

Types of Load Balancer - Based on Functions

These load balancers are classified by the way they manage and distribute network traffic. They operate at different layers (network, transport, or application) to ensure efficient request handling and high availability.

Layer 4 (L4) Load Balancer/Network Load Balancer

Layer-4 load balancers operate at the transport layer of the OSI model. They make forwarding decisions based on information available in network layer protocols (such as IP addresses and port numbers).

Key Features of Layer-4(L4) Load Balancer:

- **Transport Layer:** Operates at the transport layer (TCP/UDP).
- **Basic Load Balancing:** Distributes traffic based on IP addresses and port numbers.
- **Efficiency:** Faster processing as it doesn't inspect the content of the data packets.
- **Network Address Translation (NAT):** Can perform basic NAT to hide server addresses.

Layer 7 (L7) Load Balancer/Application Load Balancer

Layer-7 load balancers operate at the application layer of the OSI model. They can make load balancing decisions based on content, including information such as URLs, HTTP headers, or cookies.

Key Features of Layer-7(L7) Load Balancer

- **Application Layer:** Operates at the application layer (HTTP, HTTPS).
- **Content-Based Routing:** Distributes traffic based on content-specific information.
- **Advanced Routing:** Can make intelligent routing decisions based on application-specific data.
- **SSL Termination:** Capable of terminating SSL connections.

GSLB (Global Server Load Balancer) a.k.a. Multi-site Load Balancer

GSLB stands for Global Server Load Balancer. This type of load balancer goes beyond the traditional local load balancing and is designed for distributing traffic across multiple data centers or geographically distributed servers.

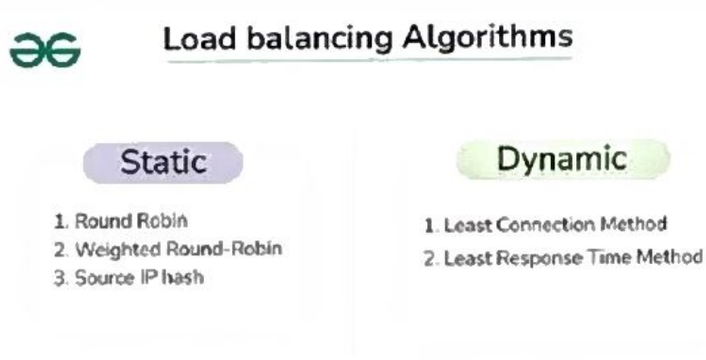
- A GSLB load balancer is concerned with global or wide-area load balancing.
- It takes into account factors such as server proximity, server health, and geographic location to intelligently distribute traffic across multiple locations.

Load Balancing Algorithms

To control traffic across servers in a network, load-balancing algorithms are important.

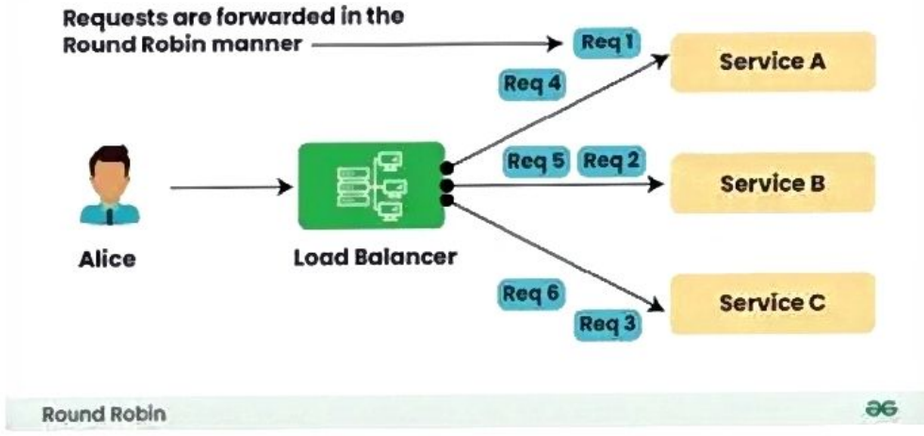
- The goal is to make sure that no single server is overloaded when too many users visit an application.

- Load balancing algorithms can be broadly categorized into two types: **Dynamic load balancing** and **Static load balancing.**



Static Load Balancing Algorithms
Static load balancing involves predetermined assignment of tasks or resources without considering real-time variations in the system. This approach relies on a fixed allocation of workloads to servers or resources, and it doesn't adapt to changes during runtime.
Types of Static Load Balancing Algorithms are:

1. Round Robin Load Balancing Algorithm
The Round Robin algorithm is a simple static load balancing approach in which requests are distributed across the servers in a sequential or rotational manner. It is easy to implement but it doesn't consider the load already on a server so there is a risk that one of the servers receives a lot of requests and becomes overloaded.
Lets say you have a group of friends, and you want to share a bag of candies equally with all of them. You give one candy to each friend in a circle, and then you start over. This is like Round Robin – making sure everyone gets a fair share.



We need to implement a basic Round Robin load balancing algorithm. The goal is to distribute incoming requests evenly among a list of servers. The first request goes to the first server, the second one goes to the second server, the third request goes to the third server and it continues further for all the requests.
Below is the implementation of the Round Robin Load Balancing Algorithm:

```
#include <iostream>
#include <vector>
#include <string>

class LoadBalancer {
private:
    std::vector<std::string> servers;
    int currentIndex;
```

```
public:
    LoadBalancer(std::vector<std::string> servers) : servers(servers), currentIndex(0) {}

    std::string getNextServer() {
        std::string nextServer = servers[currentIndex];
        currentIndex = (currentIndex + 1) % servers.size();
        return nextServer;
    }
};

int main() {
    // Sample list of servers
    std::vector<std::string> serverList = {"Server1", "Server2", "Server3"};

    // Create a load balancer with the server list
    LoadBalancer loadBalancer(serverList);

    // Simulate requests to the load balancer
    for (int i = 0; i < 10; i++) {
        std::string nextServer = loadBalancer.getNextServer();
        std::cout << "Request " << (i + 1) << ": Routed to " << nextServer << std::endl;
    }

    return 0;
}
```

```
Request 1: Routed to Server1
Request 2: Routed to Server2
Request 3: Routed to Server3
Request 4: Routed to Server1
Request 5: Routed to Server2
Request 6: Routed to Server3
Request 7: Routed to Serve...
```

Here is the explanation of the above code:

Load Balancer Class (LoadBalancer):

Maintain a list of server names.
Implement a method (getNextServer) that returns the next server in a round-robin fashion.
Keep track of the current index to determine the next server.

Main Class (RoundRobinExample):

Create a list of server names to be used in the load balancer.
Instantiate a LoadBalancer object with the list of servers.
Simulate a series of requests by repeatedly calling the getNextServer method.
Print the server to which each request is routed.

Main Class (RoundRobinExample):

Create a list of server names to be used in the load balancer.
Instantiate a LoadBalancer object with the list of servers.
Simulate a series of requests by repeatedly calling the getNextServer method.
Print the server to which each request is routed.

When to use Round Robin Load Balancing Algorithm

- Ideal for applications where all servers have similar capacity and performance.
- Works well for evenly distributed workloads, such as basic web requests.

- Best suited for simple environments without complex resource needs.
- Useful in setups where request order matters less than balanced distribution across servers.

Benefits and Drawbacks of Round Robin Load Balancing Algorithm

- **Benefits:**
  Simplicity: Easy to implement and understand.
  Fairness: Ensures that each server gets an equal share of the load.
- **Drawbacks:**
  Unequal Capacities: Doesn't consider the varying capacities of servers; treats all servers equally.
  Predictability: May not be optimal for scenarios with heterogeneous server capacities.

2. Weighted Round Robin Load Balancing Algorithm

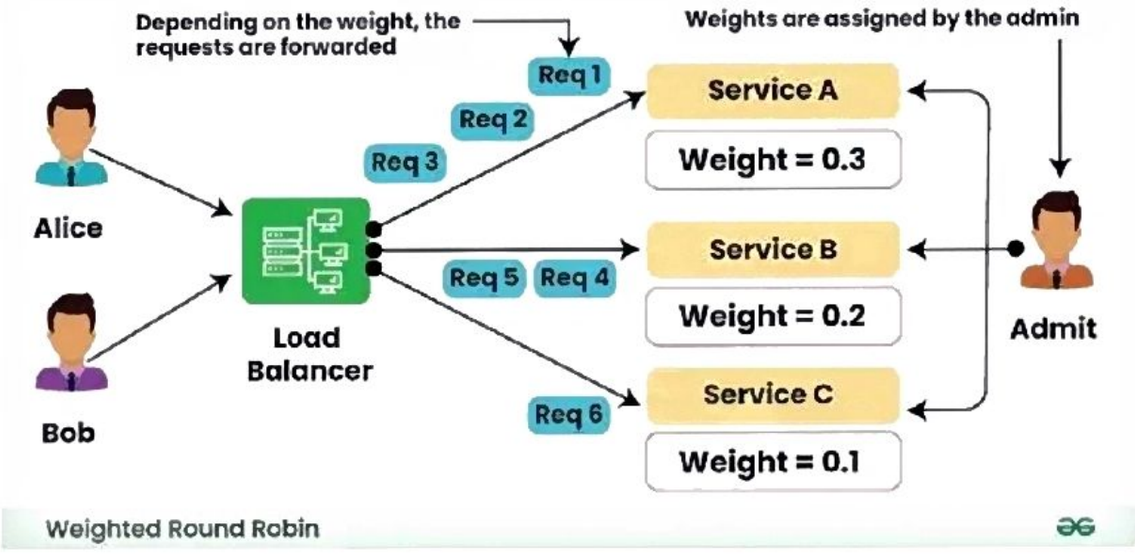
The Weighted Round Robin algorithm is also a static load balancing approach which is much similar to the round-robin technique. The only difference is, that each of the resources in a list is provided a weighted score. Depending on the weighted score the request is distributed to these servers.

- Servers with higher weights are given a larger proportion of the requests.
- The distribution is cyclic, similar to the round-robin technique, but with each server receiving a number of requests proportional to its weight.
- If a server reaches its processing capacity, it may start rejecting or queuing additional requests, depending on the server's specific behavior.

For example:

let's say your friends have different levels of candy cravings. You want to be fair, so you give more candies to the friend who loves them the most. Weighted Round Robin does something similar – it gives more tasks to the friends who can handle them better.

Let's say you have three servers with weights: **Server1 (weight 0.3)**, **Server2 (weight 0.2)**, and **Server3 (weight 0.1)**. The total weight is **0.3 + 0.2 + 0.1 = 0.6**. During each cycle, **Server1** would receive **0.3/0.6 (50%)** of the requests, **Server2** would receive **0.2/0.6 (33.33%)**, and **Server3** would receive **0.1/0.6 (16.67%).**



Below is the implementation of the Weighted Round Robin Load Balancing Algorithm:

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>

class Server {
public:
    std::string name;
```

```
    int weight;

    Server(std::string name, int weight) : name(name), weight(weight) {}
};

class WeightedRoundRobinBalancer {
private:
    std::vector<Server> servers;
    std::vector<int> cumulativeWeights;
    int totalWeight;
    int currentIndex;

    int calculateTotalWeight(const std::vector<Server>& servers) {
        int totalWeight = 0;
        for (const auto& server : servers) {
            totalWeight += server.weight;
        }
        return totalWeight;
    }

    std::vector<int> calculateCumulativeWeights(const std::vector<Server>& servers) {
        std::vector<int> cumulativeWeights(servers.size());
        cumulativeWeights[0] = servers[0].weight;
        for (size_t i = 1; i < servers.size(); i++) {
            cumulativeWeights[i] = cumulativeWeights[i - 1] + servers[i].weight;
        }
        return cumulativeWeights;
    }
public:
    WeightedRoundRobinBalancer(std::vector<Server> servers) {
        this->servers = servers;
        this->totalWeight = calculateTotalWeight(servers);
        this->cumulativeWeights = calculateCumulativeWeights(servers);
        this->currentIndex = 0;
        srand(time(0));
    }

    Server getNextServer() {
        int randomValue = rand() % totalWeight;
        for (size_t i = 0; i < cumulativeWeights.size(); i++) {
            if (randomValue < cumulativeWeights[i]) {
                currentIndex = i;
                break;
            }
        }
        return servers[currentIndex];
    }
};
```

```
int main() {
    // Sample list of servers with weights
    std::vector<Server> serverList = {
        Server("Server1", 3),
        Server("Server2", 2),
        Server("Server3", 1)
    };

    // Create a weighted round-robin load balancer with the server list
    WeightedRoundRobinBalancer balancer(serverList);

    // Simulate requests to the load balancer
    for (int i = 0; i < 10; i++) {
        Server nextServer = balancer.getNextServer();
        std::cout << "Request " << (i + 1) << ": Routed to " << nextServer.name << std::endl;
    }
    return 0;
}
```

```
Request 1: Routed to Server1
Request 2: Routed to Server2
Request 3: Routed to Server2
Request 4: Routed to Server3
Request 5: Routed to Server3
Request 6: Routed to Server3
Request 7: Routed to Serve...
```

Here is the explanation of the above code:

- **WeightedRoundRobinBalancer Class:**
  Manages a list of servers with weights.
  Calculates total and cumulative weights during initialization.
  Provides a method (getNextServer) to retrieve the next server based on weighted round-robin.
  Contains an inner class (Server) representing a server with a name and weight.
- **WeightedRoundRobinExample Class:**
  Demonstrates the usage of the WeightedRoundRobinBalancer.
  Creates a sample list of servers with weights.
  Instantiates a WeightedRoundRobinBalancer object.
  Simulates requests and prints the server to which each request is routed.

When to use Weighted Round Robin Load Balancing Algorithm?

- When servers have different capacities or performance levels.
- Ideal for environments where servers vary in resources (CPU, memory, etc.).
- Useful when you want to maximize resource utilization across all servers.
- Helps in preventing smaller servers from overloading while efficiently using larger servers.

Benefits and Drawbacks of Weighted Round Robin Load Balancing Algorithm

- **Benefits:**
  Capacity Consideration: Accounts for different server capacities by assigning weights.
  Flexibility: Can be adjusted to handle varying workloads effectively.
- **Drawbacks:**
  Complexity: More complex than simple Round Robin.
  Maintenance: Requires adjusting weights as server capacities change.

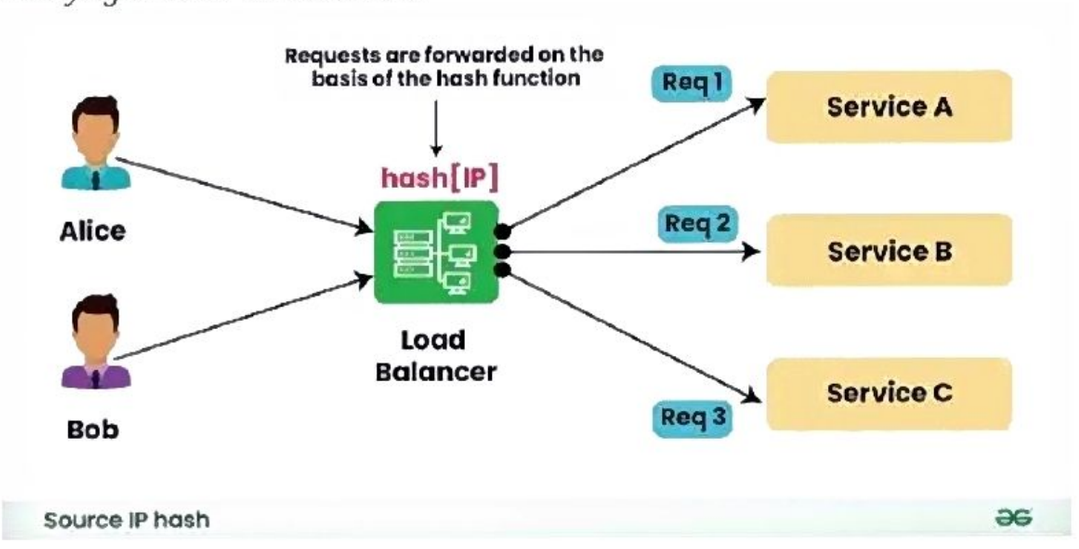
3. Source IP Hash Load Balancing Algorithm

The Source IP Hash Load Balancing Algorithm is a method used in network load balancing to distribute incoming requests among a set of servers based on the hash value of the source IP address. This algorithm aims to ensure that requests originating from the same source IP address are consistently directed to the same server.

If the load balancer is configured for session persistence, it ensures that subsequent requests from the same source IP address are consistently directed to the same server. This is beneficial for applications that require maintaining session information or state.

For example:

Think of your friends coming to your house, and you want to remember who gets which toy every time they visit. IP Hash is like remembering which friend played with which toy last time, so you always give them the same one.



We need to implement a load balancing algorithm that distributes incoming requests across a set of servers based on the hash of the source IP address. The goal is to ensure that requests coming from the same source IP address are consistently routed to the same server.

Below is the implementation of the Source IP Hash Load Balancing Algorithms:

```
#include <iostream>
#include <unordered_map>
#include <vector>
#include <string>
#include <cstdlib>

class SourceIpHashLoadBalancer {
private:
    std::unordered_map<std::string, std::string> ipToServerMap;

public:
    SourceIpHashLoadBalancer() {}

    void addServer(const std::string& serverName) {
        // Add server to the mapping
        ipToServerMap[serverName] = serverName;
    }

    std::string getServerForIp(const std::string& sourceIp) {
        // Calculate hash of the source IP
        size_t hash = std::hash<std::string>{}(sourceIp);
```

```
        // Get the list of available servers
        std::vector<std::string> servers(ipToServerMap.size());
        size_t i = 0;
        for (const auto& pair : ipToServerMap) {
            servers[i++] = pair.first;
        }

        // Map the hash value to a server index
        size_t serverIndex = hash % servers.size();

        // Return the selected server
        return servers[serverIndex];
    }
};

int main() {
    // Create a source IP hash load balancer
    SourceIpHashLoadBalancer loadBalancer;

    // Add servers to the load balancer
    loadBalancer.addServer("Server1");
    loadBalancer.addServer("Server2");
    loadBalancer.addServer("Server3");

    // Simulate requests with different source IPs
    std::vector<std::string> sourceIps = {"192.168.1.1", "10.0.0.1", "172.16.0.1"};

    for (const std::string& sourceIp : sourceIps) {
        std::string selectedServer = loadBalancer.getServerForIp(sourceIp);
        std::cout << "Request from " << sourceIp << " routed to " << selectedServer << std::endl;
    }

    return 0;
}
```

Request from 192.168.1.1 routed to Server2
Request from 10.0.0.1 routed to Server3
Request from 172.16.0.1 routed to Server1

Here is the explanation of the above code:

- **SourceIpHashLoadBalancer Class:**
  Fields: ipToServerMap, a mapping of server names to server names (used for consistent hashing).
  Methods: addServer, adds a server to the load balancer; getServerForIp, calculates the hash of the source IP and determines the server to handle the request.
- **SourceIpHashLoadBalancerExample Class:**
  Demonstrates the usage of the SourceIpHashLoadBalancer.
  Creates an instance of the load balancer, adds servers, and simulates requests from different source IPs.

When to use Source IP Hash Load Balancing Algorithm?

- Ideal for applications needing session consistency, like online banking, where the same user must connect to the same server throughout a session.
- Useful when users from specific regions should connect to dedicated servers for better performance or compliance.
- Effective when a few IPs generate most of the traffic, ensuring balanced load distribution without random switching.

Benefits and Drawbacks of Source IP Hash Load Balancing Algorithm:

- **Benefits:**
  Consistency: Ensures requests from the same source IP always go to the same server, maintaining session state.
  Predictability: Useful when connection persistence is critical.
- **Drawbacks:**
  Limited Distribution: May lead to uneven load distribution if certain source IPs are more active.
  Scaling Challenges: Adding or removing servers may disrupt session persistence.

Dynamic Load Balancing Algorithms

Dynamic load balancing involves making real-time decisions about how to distribute incoming network traffic or computational workload across multiple servers or resources. This approach adapts to the changing conditions of the system, such as variations in server load, network traffic, or resource availability.

The choice between dynamic and static load balancing depends on the characteristics of the system, the nature of the workload, and the desired level of adaptability. Dynamic load balancing is often favored in dynamic, high-traffic environments, while static load balancing may be suitable for more predictable scenarios.

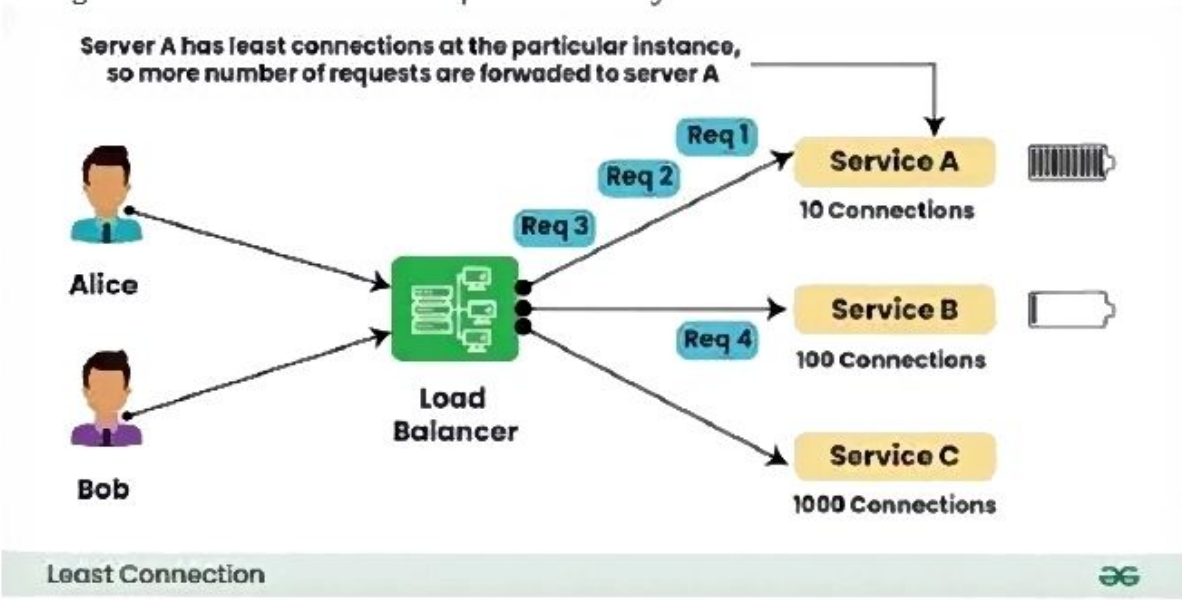
1. Least Connection Method Load Balancing Algorithm

The Least Connections algorithm is a dynamic load balancing approach that assigns new requests to the server with the fewest active connections. The idea is to distribute incoming workloads in a way that minimizes the current load on each server, aiming for a balanced distribution of connections across all available resources.

- To do this load balancer needs to do some additional computing to identify the server with the least number of connections.
- This may be a little bit costlier compared to the round-robin method but the evaluation is based on the current load on the server.

For example:

Lets say you're at a playground, and some kids are playing on different swings. You want to join the swing with the fewest kids so that it's not too crowded. Least Connection is like choosing the swing with the least number of kids already on it.



Least Connection

We need to implement a Load Balancing Algorithm that distribute incoming requests across a set of servers and should aim to minimize the number of active connections on each server by directing

new requests to the server with the fewest active connections. This ensures a balanced distribution of the workload and prevents overload on individual servers.

Below is the implementation of the Least Connection Method Load Balancing Algorithms:

```
#include <map>
#include <string>
#include <limits>
#include <iostream>

class LeastConnectionLoadBalancer {
private:
    std::map<std::string, int> serverConnections;

public:
    LeastConnectionLoadBalancer() {}

    void addServer(const std::string& serverName) {
        serverConnections[serverName] = 0;
    }

    std::string getServerWithLeastConnections() {
        int minConnections = std::numeric_limits<int>::max();
        std::string selectedServer;

        for (const auto& entry : serverConnections) {
            if (entry.second < minConnections) {
                minConnections = entry.second;
                selectedServer = entry.first;
            }
        }

        if (!selectedServer.empty()) {
            serverConnections[selectedServer] = minConnections + 1;
        }

        return selectedServer;
    }
};

int main() {
    LeastConnectionLoadBalancer loadBalancer;

    loadBalancer.addServer("Server1");
    loadBalancer.addServer("Server2");
    loadBalancer.addServer("Server3");

    for (int i = 0; i < 10; i++) {
        std::string selectedServer = loadBalancer.getServerWithLeastConnections();
        std::cout << "Request " << (i + 1) << ": Routed to " << selectedServer << std::endl;
    }
```

```
    return 0;
}
```

```
Request 1: Routed to Server1
Request 2: Routed to Server2
Request 3: Routed to Server3
Request 4: Routed to Server1
Request 5: Routed to Server2
Request 6: Routed to Server3
Request 7: Routed to Serve...
```

Here is the explanation of the above code:

- **LeastConnectionLoadBalancer Class**
  Fields:
- serverConnections: Tracks active connections per server.

Methods:

- addServer: Adds a server with 0 initial connections.
- getServerWithLeastConnections: Returns server with fewest connections and increments its count.
- **LeastConnectionLoadBalancerExample Class**
  Main Method:
- Create LeastConnectionLoadBalancer instance.
- Add servers.
- Simulate requests and route to server with least connections.

When to use Least Connection Load Balancing Algorithm?

- Ideal for applications where some requests take longer to process than others (e.g., video streaming or large file uploads).
- Useful when some connections stay active longer, as it ensures new requests go to servers with fewer active connections.
- Great for systems with fluctuating traffic, as it balances based on real-time server load rather than just counting requests.

Benefits and Drawbacks of Least Connection Load Balancing Algorithm:

Benefits:

Balanced Load: Distributes traffic to servers with the fewest active connections, preventing overloading.

Dynamic: Adapts to changing server workloads.

Drawbacks:

Ignored Capacities: Ignores server capacities; a server with fewer connections may still have less capacity.

Sticky Sessions: May not be suitable for scenarios requiring session persistence.

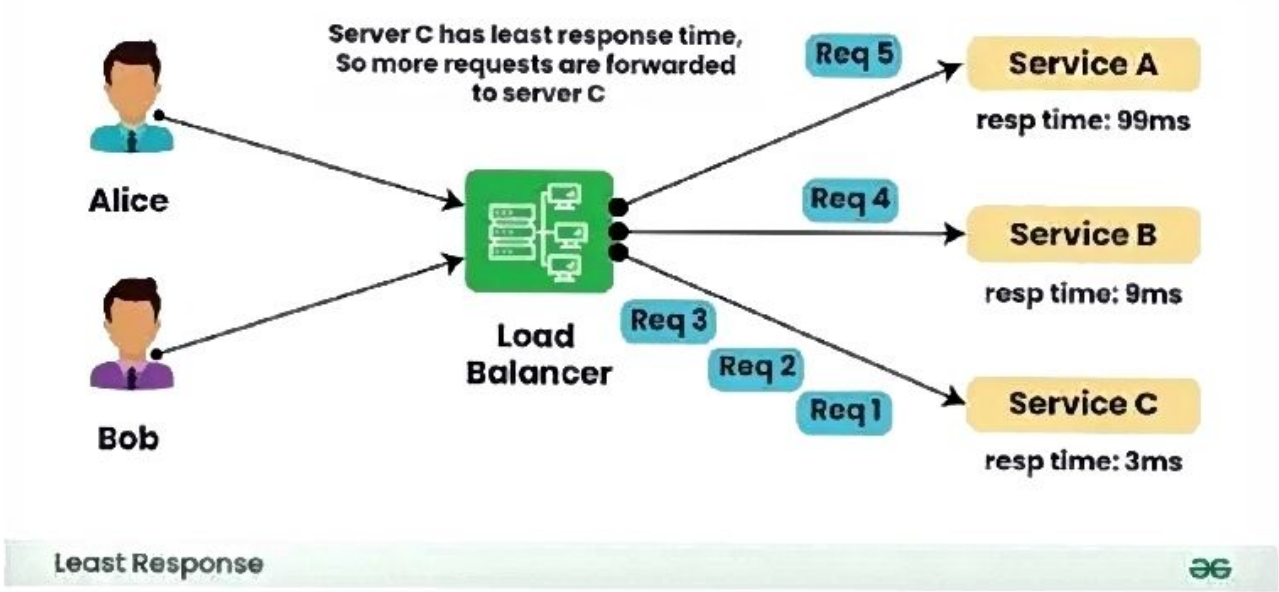
2. Least Response Time Method Load Balancing Algorithm

The Least Response method is a dynamic load balancing approach that aims to minimize response times by directing new requests to the server with the quickest response time.

- It considers the historical performance of servers to make decisions about where to route incoming requests, optimizing for faster processing.
- The dynamic aspect comes from the continuous monitoring of server response times and the adaptive nature of the algorithm to route incoming requests to the server with the historically lowest response time.

For example:

Picture yourself at a snack bar where you can order food from different servers. You notice that some servers are faster than others. You choose the server that seems to serve food the quickest each time you go. Least Response Time is like picking the server with the shortest line.



Least Response

We need to implement a Load Balancing Algorithm that distribute incoming requests across a set of servers and should aim to minimize the response time by directing new requests to the server with the least accumulated response time. This ensures a balanced distribution of the workload and helps optimize the overall system performance.

Below is the implementation of the Least Response Time Load Balancing Algorithms:

```
#include <map>
#include <string>
#include <limits>
#include <iostream>

class LeastResponseLoadBalancer {
private:
    std::map<std::string, long long> serverResponseTimes;

public:
    LeastResponseLoadBalancer() {}

    void addServer(const std::string& serverName) {
        serverResponseTimes[serverName] = 0;
    }

    std::string getServerWithLeastResponseTime() {
        long long minResponseTime = std::numeric_limits<long long>::max();
        std::string selectedServer;

        for (const auto& entry : serverResponseTimes) {
            if (entry.second < minResponseTime) {
                minResponseTime = entry.second;
                selectedServer = entry.first;
            }
        }

        if (!selectedServer.empty()) {
            serverResponseTimes[selectedServer] = minResponseTime + 1;
        }
```

```
        return selectedServer;
    }
};

int main() {
    LeastResponseLoadBalancer loadBalancer;

    loadBalancer.addServer("Server1");
    loadBalancer.addServer("Server2");
    loadBalancer.addServer("Server3");

    for (int i = 0; i < 10; i++) {
        std::string selectedServer = loadBalancer.getServerWithLeastResponseTime();
        std::cout << "Request " << (i + 1) << ": Routed to " << selectedServer << std::endl;
    }

    return 0;
}
```

```
Request 1: Routed to Server1
Request 2: Routed to Server2
Request 3: Routed to Server3
Request 4: Routed to Server1
Request 5: Routed to Server2
Request 6: Routed to Server3
Request 7: Routed to Serve...
```

Here is the explanation of the above code:

LeastResponseLoadBalancer Class:

Fields:

serverResponseTimes: A map that tracks the accumulated response time for each server.

Methods:

addServer: Adds a server to the load balancer with an initial response time of 0.

getServerWithLeastResponseTime: Determines the server with the least accumulated response time and increments its response time.

LeastResponseLoadBalancerExample Class:

Main Method:

Creates an instance of the LeastResponseLoadBalancer.

Adds servers to the load balancer.

Simulates requests and prints the server to which each request is routed based on the least response time algorithm.

When to use Least Response Time Load Balancing Algorithm?

- Ideal for applications with heavy, fluctuating user traffic where response time matters.
- Great for apps like e-commerce sites or streaming services, where a quick response improves user experience.
- Works well when servers have different load levels, as it directs traffic to the server that's both available and responds the fastest.

Benefits and Drawbacks of Least Response Time Load Balancing Algorithm:

Benefits:

Optimized Performance: Directs traffic to servers with the quickest response times, optimizing overall system performance.

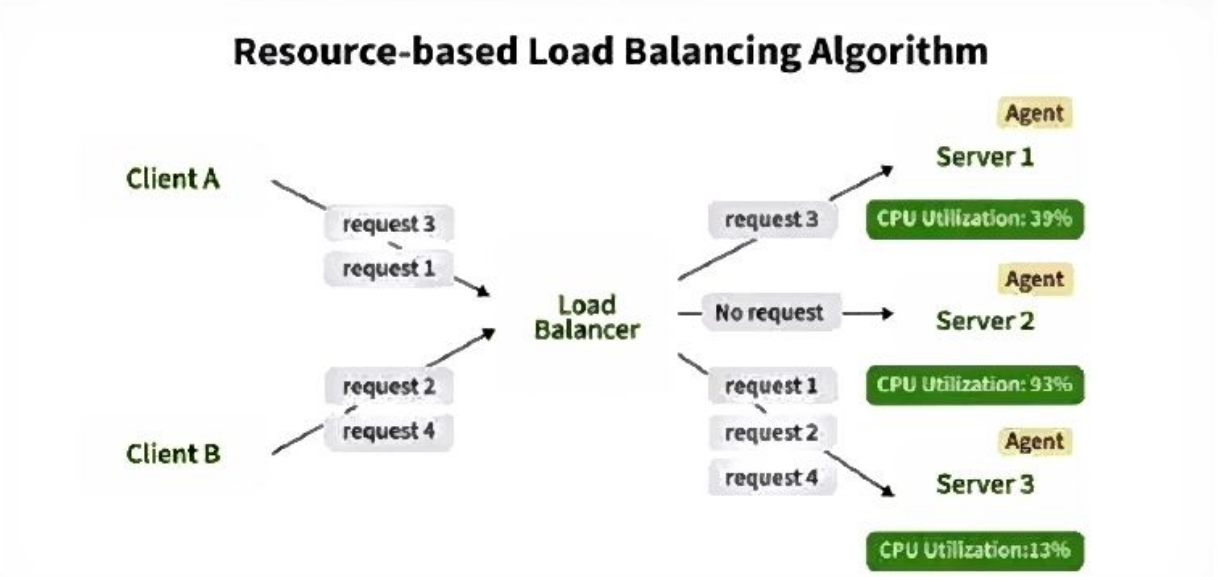
Adaptable: Adjusts to changes in server responsiveness over time.
Drawbacks:
Historical Bias: Heavily influenced by past response times, may not always reflect current server capabilities.
Complex Implementation: Requires tracking and managing historical response times.

3. Resource-based Load Balancing Algorithm

The Resource-Based Load Balancing algorithm distributes incoming requests based on the current resource availability of each server, such as CPU usage, memory, or network bandwidth. Rather than just routing traffic equally or based on past performance, this algorithm evaluates the current "resource health" of each server to decide where new requests should go.
Resource-based Load Balancing Algorithm

For example:

Imagine it like assigning tasks in an office based on each employee's workload at the moment—some are busy, while others are free. Resource-Based Load Balancing directs requests to the server with the most available resources.



To implement Resource-Based Load Balancing, you need to track each server's current resources, then route each new request to the server that can handle it best based on real-time data. Here's an example implementation of a Resource-Based Load Balancing Algorithm. This simple implementation uses CPU load as a metric to decide which server should handle each request.

```
#include <iostream>
#include <map>
#include <limits>

class Server {
public:
   std::string name;
   double cpuLoad;  // Represents the current CPU load percentage of the server

   Server(std::string name) {
      this->name = name;
      this->cpuLoad = 0.0;
   }

   // Simulate updating the CPU load for the server
   void updateCpuLoad(double load) {
      this->cpuLoad = load;
   }
```

```
    double getCpuLoad() {
        return this->cpuLoad;
    }

    std::string getName() {
        return this->name;
    }
};

class ResourceBasedLoadBalancer {
private:
    std::map<std::string, Server> servers;

public:
    ResourceBasedLoadBalancer() {}

    // Adds a server to the load balancer
    void addServer(Server server) {
        servers[server.getName()] = server;
    }

    // Finds the server with the lowest CPU load and assigns the request to it
    Server getServerWithMostResources() {
        Server bestServer;
        double lowestLoad = std::numeric_limits<double>::max();

        for (auto& pair : servers) {
            Server server = pair.second;
            if (server.getCpuLoad() < lowestLoad) {
                lowestLoad = server.getCpuLoad();
                bestServer = server;
            }
        }
        return bestServer;
    }

    // Simulate handling a request
    void handleRequest() {
        Server bestServer = getServerWithMostResources();
        if (bestServer.getName() != "") {
            std::cout << "Routing request to server: " << bestServer.getName() << " with current CPU
load: " << bestServer.getCpuLoad() << "%\n";
            // Here, you might update the server's load after handling a request
        } else {
            std::cout << "No servers available.\n";
        }
    }
};
```

```
int main() {
    ResourceBasedLoadBalancer loadBalancer;

    // Create servers and add them to the load balancer
    Server server1("Server1");
    Server server2("Server2");
    Server server3("Server3");

    loadBalancer.addServer(server1);
    loadBalancer.addServer(server2);
    loadBalancer.addServer(server3);

    // Simulate updating CPU load for each server
    server1.updateCpuLoad(30.0);
    server2.updateCpuLoad(50.0);
    server3.updateCpuLoad(20.0);

    // Route requests based on current CPU load
    loadBalancer.handleRequest();  // This should route to Server3, as it has the lowest CPU load
    return 0;
}
```

Output

```
Routing request to server: Server3 with current CPU load: 20.0%
```

Below is the explanation of the above code:

- **Server Class:**
  Represents a server with attributes like name and cpuLoad. The updateCpuLoad method updates the CPU load to simulate real-time monitoring.
- **ResourceBasedLoadBalancer Class:**
  Manages servers and routes requests based on available resources. addServer adds a server. getServerWithMostResources finds the server with the lowest CPU load. handleRequest routes requests to the server with the most available resources..
- **ResourceBasedLoadBalancerExample Class:**
  Sets up the load balancer, adds servers, updates CPU loads, and simulates routing requests.

When to Use Resource-Based Load Balancing Algorithm?

- Useful for applications that perform CPU-intensive or memory-heavy tasks.
- Works well when servers have different resource levels, as the algorithm adapts to each server's real-time capacity.
- Ensures availability by routing requests to the least overloaded servers, reducing downtime risks.

Benefits and Drawbacks of Resource-Based Load Balancing Algorithm

- **Benefits:**
  Resource Optimization: Balances workloads based on real-time resource data, improving system efficiency.
  Adaptability: Adjusts dynamically to the current state of each server's resources.
- **Drawbacks:**
  Complex Implementation: Requires continuous monitoring of server resources, which can add complexity.

Higher Overhead: Real-time tracking of resources may consume additional system resources.